Memory Residuals

Yueze Liu
Independent Researcher
yuezel2@illinois.edu

Call for Collaboration
Seeking Academic Partnership

April 26, 2026

Abstract

Modern Large Language Models (LLMs) deployed as conversational agents are constrained by their inability to maintain lifelong, multi-session memory. Current paradigms rely on either concatenating ever-expanding chat histories (leading to context dilution and massive compute overhead) or bolting on external Vector-DB Retrieval-Augmented Generation (RAG) and text summarization systems. Both approaches lack innate cognitive routing—the ability to implicitly choose *what* to remember from the past and *when* to actively recall it. Drawing inspiration from human episodic memory, we propose Memory Residuals. By forcing multi-turn conversation logs through a recurrent, learnable summarizing memory block, we create a dense latent state of user persona and history. Rather than appending this state to the sequence context window, we inject it directly into the depth-wise residual stream, allowing models to seamlessly bypass memory during standard dialogue and sample it heavily during historical callbacks. This position paper outlines the mathematical formulation of Memory Residuals as an elegant replacement for rigid external state-tracking, and calls for collaborative empirical validation.

1 Introduction

The current trajectory of memory scaling in Transformer-based architectures [1] fails to address the unique demands of lifelong conversational AI. To give agents persistent memory across thousands of distinct conversational turns, developers either brute-force the context window (expanding sequence length N and suffering from $O(N^2)$ scaling limits), or they rely on rigid, non-differentiable workarounds like text-summarization loops and external Retrieval-Augmented Generation (RAG) databases [3].

These approaches are fundamentally inefficient. Appending 50 past chat sessions to a system prompt is computationally wasteful for a simple greeting turn, and semantic search often fails to capture evolving implicit user preferences. Human cognition does not perfectly retain raw conversational transcripts, nor does it search a disconnected database to maintain a relationship. Instead:

1. **Human episodic memory is finite and latent** (compressing raw dialogue into dense, high-level semantic abstractions and state).
2. **Human memory retrieval is top-down and attention-driven**; we innately route our cognition, bypassing memory for casual conversational filler, and heavily querying past interactions by dynamically weighing them into our active thoughts.

We argue that this dynamic, information-choosing behavior should be baked natively into the architecture. In this paper, we propose a mechanism that merges finite latent space compression with recent advancements in depth-wise attention routing [4], creating a persistent, differentiable memory state for multi-session AI.

2 The Core Paradigm: Memory Residuals

We introduce **Memory Residuals**, an end-to-end differentiable working memory mechanism. It operates by breaking down lifelong conversational tracking into two key architectural advances: the recurrent, incremental construction of a latent episodic memory block, and the dynamic, single-pass querying of that block via the model's internal residual stream.

2.1 Key Advance 1: Recurrent Updates via Two-Stage QKV Competition

A naive approach to memory compression might attempt to attend over all historical tokens at once, but this recreates the very $O(N^2)$ context limits we seek to avoid. Furthermore, compressing thousands of conversational turns into a fixed latent bottleneck risks catastrophic forgetting, where conversational filler from a new session overwrites critical persona data established sessions ago.

To resolve this, we formulate the episodic memory block as an incrementally updated matrix $M_{\mathrm{c}} \in \mathbb{R}^{K \times d}$ (where K is a small, fixed number of latent vectors, e.g., 128, and d is the hidden dimension). We update this state using a **Two-Stage QKV Pipeline**, separating the tasks of information *extraction* and information *judgment*.

Let $M_{\mathrm{c}}^{t-1} \in \mathbb{R}^{K \times d}$ be the persistent latent memory state prior to session t, and let $C_t \in \mathbb{R}^{N_t \times d}$ be the raw tokens of the newly completed session.

Stage 1: The Extraction Layer (Compression) First, we compress the local session into a dense candidate state $E_t \in \mathbb{R}^{K \times d}$. Using a set of fixed, learnable extraction queries M_{in}, the model pulls only the salient features from the raw session tokens:

$$E_t = \mathrm{Softmax}\left(\frac{M_{\mathrm{in}} W_Q^{\mathrm{ext}} (C_t W_K^{\mathrm{ext}})^\top}{\sqrt{d}}\right) C_t W_V^{\mathrm{ext}} \tag{1}$$

Stage 2: The Judging Layer (QKV Competition) We now possess two identically sized latent matrices representing the old knowledge (M_{c}^{t-1}) and the newly extracted candidate knowledge (E_t). We concatenate them into a combined judgment pool $P_{\mathrm{judge}} = [M_{\mathrm{c}}^{t-1} \parallel E_t] \in \mathbb{R}^{2K \times d}$.

To update the memory, we employ a second cross-attention block governed by a distinct set of persistent, learnable judging queries $M_{\mathrm{judge}} \in \mathbb{R}^{K \times d}$. These queries evaluate the combined pool:

$$M_{\mathrm{c}}^{t} = \mathrm{Softmax}\left(\frac{M_{\mathrm{judge}} W_Q^{\mathrm{judge}} (P_{\mathrm{judge}} W_K^{\mathrm{judge}})^\top}{\sqrt{d}}\right) P_{\mathrm{judge}} W_V^{\mathrm{judge}} \tag{2}$$

The Forgetting Defense Insight: Because the Softmax is computed across the $2K$ dimension, the old memory and the new candidate memory are forced into a zero-sum semantic competition. If the new session C_t contains only mundane conversational filler, E_t will lack predictive semantic value. The Judging Queries will recognize this and shift their attention distribution heavily onto the keys of M_{c}^{t-1}, effectively routing the old memory untouched into the next state. Conversely, if E_t contains a critical state-change, the queries will strongly attend to the new representation, implicitly overwriting outdated information.

2.1.1 On Extraction Depth: Multi-Layer Distillation as an Ablation Axis

Equation 1 assumes that a single cross-attention pass over C_t is sufficient to distill the session's salient semantic content into a K-slot candidate. In practice, deciding *what to extract* from a raw conversation of hundreds of tokens is a highly non-linear task: a throwaway mention of "my sister" early in the session may only surface as meaningful after composing it with later turns of the same session. The judging step that follows operates on a $2K$-slot pool and is therefore

already a low-bandwidth decision; the capacity we want is in the distillation that *feeds* that decision.

We therefore identify **extraction depth** L_E as a primary ablation axis, replacing Eq. 1 with a Perceiver-style refinement stack in which the latent state iteratively re-queries the raw session:

$$E^{(0)} = \mathrm{CrossAttn}^{(0)}(M_{\text{in}},\ C_t) \in \mathbb{R}^{K\times d} \tag{3}$$

$$E^{(\ell)} = E^{(\ell-1)} + \mathrm{CrossAttn}^{(\ell)}\Big(E^{(\ell-1)},\ C_t\Big) \in \mathbb{R}^{K\times d}, \quad \ell = 1,\dots,L_E \tag{4}$$

The final candidate $M_{\text{new}} := E^{(L_E)}$ replaces E_t in the judging step of Eq. 2, which remains a single zero-sum competition:

$$M_{\text{c}}^t = \mathrm{Softmax}\left(\frac{M_{\text{judge}}W_Q^{\text{judge}}\left(P_{\text{judge}}W_K^{\text{judge}}\right)^\top}{\sqrt{d}}\right)P_{\text{judge}}W_V^{\text{judge}}, \quad P_{\text{judge}} = \left[M_{\text{c}}^{t-1} \parallel M_{\text{new}}\right] \tag{5}$$

Because every intermediate state in the refinement lives in $\mathbb{R}^{K\times d}$, the residual in Eq. 4 is well-typed and $L_E = 0$ trivially recovers the single-layer baseline. Deeper stacks ($L_E = 2$–8) provide non-linear refinement capacity for multi-hop extraction, but must be empirically justified against the risk of overfitting to training-distribution dialogue patterns.

2.2 Key Advance 2: Implicit Retrieval via the Depth-Wise Residual Stream

Our second advance defines how the active model queries the persistent state M_{c}^t during a live conversation. Treating memory retrieval as a standard token-level attention operation over a concatenated pool would reintroduce an $O((N+K)^2)$ bottleneck. To avoid sequence-length bloat, we register the episodic memory block directly into the model's *depth-wise* residual stream, utilizing the principles of Attention Residuals [4].

Reading Memory as a Source Stream. Depth-wise routing frameworks treat each layer's output as a source tensor of shape $S \times d$: one d-dimensional vector per token position in the current session. To interface seamlessly with this structure—and to let each position in the current session retrieve the memory slot most relevant to its local context—we read out from the K-row memory matrix M_{c}^t via a cross-attention whose queries are the current session's token embeddings $X \in \mathbb{R}^{S\times d}$:

$$m^t = \mathrm{Softmax}\left(\frac{XW_Q^{\text{read}}(M_{\text{c}}^t W_K^{\text{read}})^\top}{\sqrt{d}}\right)M_{\text{c}}^t W_V^{\text{read}} \in \mathbb{R}^{S\times d} \tag{6}$$

Each position s attends over the K memory slots independently, so the readout is inherently query-dependent: tokens discussing a family conflict route mass onto slots encoding family context, while tokens about dinner plans attend elsewhere. Because m^t has exactly the shape of a standard attention layer's output, it drops directly into the depth-wise routing pool described below without any special-casing.

Why Block AttnRes, Not Full AttnRes. A naive integration would make m^t a foundational source for a *full* depth-wise softmax over all $L-1$ preceding layer outputs. While conceptually clean, Full AttnRes incurs $O(Ld)$ per-token memory and—under pipeline parallelism—$O(Ld)$ cross-stage communication, which the AttnRes authors flag as the dominant scaling obstacle at frontier model sizes [4]. We therefore adopt the *Block* variant: the L transformer layers are partitioned into N blocks of $S_B = L/N$ layers each, with intra-block accumulation reduced to a standard summation and inter-block aggregation handled by softmax attention over

only the N block-level summaries. This collapses depth-wise memory and communication from $O(Ld)$ to $O(Nd)$ ($N \approx 8$ in practice), making memory-augmented depth routing a practical drop-in for standard residual connections at scale.

Registering Memory as a Foundational Block Source. Let $\mathcal{B}_n$ denote the set of layer indices in block n, for $n = 1, \ldots, N$. Following [4], the intra-block partial sum after the first i layers of block n is:

$$b_n^i = \sum_{j \in \mathcal{B}_n^{1:i}} f_j(h_j) \in \mathbb{R}^{S \times d}, \qquad b_n := b_n^{S_B} \quad \text{(completed block representation)} \tag{7}$$

The token embedding occupies the conventional foundational slot $b_0 := h_1$. We register the per-position memory readout as a parallel *external* foundational source, prepended to the block pool:

$$b_{-1} := m^t \quad \in \mathbb{R}^{S \times d} \tag{8}$$

This index choice keeps the AttnRes recurrence (which sets $b_0 = h_1$) intact while exposing lifelong episodic memory as a sibling foundational source that every subsequent layer in every block can query.

Inter-Block Attention with Memory. For the i-th layer of block n, the value pool aggregates the memory source, the embedding, all completed prior block summaries, and—for layers beyond the first in the current block—the running intra-block partial sum:

$$\mathcal{V}_{n,i} = \begin{cases} \big[\, b_{-1},\ b_0,\ b_1,\ \ldots,\ b_{n-1} \,\big] & i = 1, \\ \big[\, b_{-1},\ b_0,\ b_1,\ \ldots,\ b_{n-1},\ b_n^{i-1} \,\big] & i \geq 2. \end{cases} \tag{9}$$

The input to the i-th layer of block n is then a softmax attention over this pool:

$$h_{n,i} = \sum_{v \in \mathcal{V}_{n,i}} \alpha_{v \to (n,i)} \cdot v, \qquad \alpha_{v \to (n,i)} = \frac{\phi(w_{n,i},\, v)}{\sum_{v' \in \mathcal{V}_{n,i}} \phi(w_{n,i},\, v')} \tag{10}$$

where $\phi(q, v) = \exp\big(q^\top \operatorname{RMSNorm}(v)\big)$ and $w_{n,i} \in \mathbb{R}^d$ is a learned pseudo-query for the i-th layer of block n, initialized to zero so that initial weights $\alpha_{v \to (n,i)}$ are uniform across sources [4]. Setting $N = L$ (one layer per block) recovers Full AttnRes, while $N = 1$ degenerates to a standard residual stream with b_{-1} and b_0 isolated as the only non-trivial sources. The pool size is $|\mathcal{V}_{n,i}| \in \{n{+}1,\, n{+}2\}$, bounded by $N{+}2$ regardless of L.

Implicit Cognitive Routing. By integrating memory directly into the block-wise residual stream, the model gains the ability to implicitly control memory recall without gating heuristics. During a content-rich conversational turn, the recent block summaries $(b_1, \ldots, b_{n-1})$ and the running partial sum b_n^{i-1} will naturally dominate the softmax competition, suppressing $\alpha_{b_{-1} \to (n,i)}$ and bypassing historical memory. Conversely, during a sparse or ambiguous turn (e.g., a bare pronoun callback like "Did you ask her yet?"), the absence of dominant local features allows the memory source $b_{-1} = m^t$ to attract higher attention weight. This structurally replicates the selective, top-down nature of human cognitive retrieval, while the block partition keeps the cross-stage communication budget independent of model depth.

3 Distinction from Existing Paradigms

While our approach shares philosophical DNA with several recent works, it uniquely resolves the limitations of modern agentic workflows:

- **Versus Standard RAG and MemGPT:** Memory Residuals are entirely end-to-end differentiable. The model learns *how* to encode implicit user preferences based on backpropagated loss, eliminating the need for rigid vector databases and lossy text-summarization prompts.
- **Versus Attention Residuals [4]:** While recent work replaces uniform depth-wise addition with softmax attention over *internal* layer outputs (and its scalable Block AttnRes variant over N block-level summaries), Memory Residuals expand this paradigm to the *external* temporal domain. By prepending the per-position cross-session readout m^t as a sibling foundational source (b_{-1}) inside the same Block AttnRes pool used for intra-session depth routing, we bridge the gap between depth-wise aggregation and lifelong memory retrieval while preserving both the query-dependent selectivity of standard attention and the $O(Nd)$ depth-wise communication budget that makes the mechanism practical at scale.
- **Versus Perceiver IO [5]:** While Perceiver uses cross-attention bottlenecks to handle large inputs, Memory Residuals embed this bottleneck specifically as a *softmax-weighted depth source*, allowing context-dependent querying on a turn-by-turn basis.
- **Versus Recurrent Memory Transformer (RMT) [6]:** Models like RMT prepend memory embeddings directly to the context window. Memory Residuals remove memory from the standard $O(N^2)$ sequence entirely, isolating it in the orthogonal depth-wise routing pool.

4 Empirical Roadmap: Training via Episodic TV Scripts

A significant bottleneck in lifelong conversational AI research is the lack of high-quality, long-horizon dialogue data exhibiting true implicit memory recall. Synthetic datasets lack emotional depth, and real-world assistant logs are frequently restricted by privacy constraints.

To empirically validate Memory Residuals, we propose utilizing **Episodic TV Show Transcripts** (e.g., sitcoms and long-form dramas) as our primary training and benchmarking corpus. TV scripts represent the gold standard of multi-session human dialogue. They are defined by stable character personas, long contexts, and explicitly scripted "callbacks" (e.g., a thematic reference in Season 3 that relies entirely on a brief interaction from Season 1).

By feeding transcripts episode-by-episode into the model and updating M_{c} sequentially, we can evaluate the model's ability to execute a semantic callback across massive token spans. Successfully retrieving the context of a Season 1 event during a Season 3 generation—while ignoring hundreds of intervening episodes of casual filler—will empirically prove that the Memory Residuals mechanism accurately mimics human selective recall.

5 Conclusion and Call for Collaboration

The Memory Residuals architecture presents a mathematically elegant solution to the exploding context constraints and rigid external memory systems of multi-turn AI agents. By combining a recurrent, competition-driven episodic memory block with depth-wise residual routing, we provide a blueprint for models that innately understand when to ignore the past, and when to actively retrieve it.

I have established the theoretical framework and foundational mathematics for this approach. I am currently seeking an academic laboratory or partner with compute resources to collaborate on scaling this architecture. Future work will execute the TV-script training curriculum to validate the activation of the memory attention weights (α), with the ultimate goal of co-authoring a formal publication for upcoming AI conferences (e.g., NeurIPS, ICLR).

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need.* Advances in Neural Information Processing Systems, 30.

[2] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). *Lost in the middle: How language models use long contexts.* Transactions of the Association for Computational Linguistics, 12, 157–173.

[3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks.* Advances in Neural Information Processing Systems, 33, 9459–9474.

[4] Kimi Team (Du, Y., et al.). (2026). *Attention Residuals.* arXiv preprint arXiv:2603.15031.

[5] Jaegle, A., Borgeaud, S., Alayrac, J.-B., Doersch, C., Ionescu, C., Ding, D., Koppula, S., Zoran, D., Brock, A., Shelhamer, E., & Hénaff, O. (2021). *Perceiver IO: A General Architecture for Structured Inputs & Outputs.* International Conference on Learning Representations (ICLR).

[6] Bulatov, A., Kuratov, Y., & Burtsev, M. S. (2022). *Recurrent Memory Transformer.* Advances in Neural Information Processing Systems, 35, 11079–11091.

[7] Chevalier, A., Wettig, A., Ajith, A., & Chen, D. (2023). *Adapting Language Models to Compress Contexts.* Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP).